

1 Kafka Basics (Foundational – Freshers / 1–2 yrs)

1. What is Apache Kafka and why is it used?
 2. What problem does Kafka solve compared to traditional messaging systems?
 3. Explain **Producer, Broker, Consumer**.
 4. What is a **Topic**?
 5. What is a **Partition** and why is it important?
 6. What is **Offset** in Kafka?
 7. Difference between **Kafka and RabbitMQ**.
 8. What is a **Consumer Group**?
 9. What happens if a consumer in a group fails?
 10. Why is Kafka called a **distributed commit log**?
-

2 Kafka Core Concepts (Intermediate – 2–4 yrs)

11. How does Kafka achieve **high throughput**?
 12. Explain **replication factor**.
 13. What is a **Leader and Follower** partition?
 14. What is **ISR (In-Sync Replica)**?
 15. What happens if the **leader broker fails**?
 16. Explain **acks=0, 1, all**.
 17. What is **log retention**?
 18. Difference between **log retention time vs size**.
 19. What is **Zookeeper's role** in Kafka?
 20. What happens when Zookeeper goes down?
-

3 Producers & Consumers (Intermediate)

21. How does a Kafka producer decide which partition to send data to?
22. What is a **key** in Kafka messages?
23. What is **idempotent producer**?
24. What are **consumer commit strategies**?
25. Difference between **auto commit and manual commit**.
26. What is **poll()** method?

- 27. What is **rebalance**?
 - 28. How do you handle **consumer lag**?
 - 29. What happens if a consumer processes data slowly?
 - 30. How to ensure **at-least-once** delivery?
-

4 Message Delivery Semantics

- 31. Explain **at-most-once** delivery.
 - 32. Explain **at-least-once** delivery.
 - 33. Explain **exactly-once** semantics.
 - 34. How does Kafka implement **exactly-once**?
 - 35. What is **transactional producer**?
 - 36. Difference between **idempotent producer** vs **transactional producer**.
-

5 Kafka Internals (Advanced – 4–7 yrs)

- 37. How does Kafka store messages on disk?
 - 38. What is **segment file**?
 - 39. Explain **index and time-index files**.
 - 40. How does Kafka ensure **message ordering**?
 - 41. What is **zero-copy** optimization?
 - 42. How does Kafka handle **backpressure**?
 - 43. Explain **controller broker**.
 - 44. How metadata is shared across brokers?
 - 45. What happens during **leader election**?
-

6 Kafka Performance & Tuning

- 46. How do you increase Kafka throughput?
- 47. Producer tuning parameters you have used.
- 48. Consumer tuning parameters.
- 49. How to reduce **latency**?
- 50. What causes **high consumer lag**?
- 51. How do you monitor Kafka performance?

- 52. Important Kafka metrics.
 - 53. How many partitions should a topic have?
 - 54. Impact of too many partitions.
 - 55. How to handle **hot partitions**?
-

7 Kafka Streams & Connect

- 56. What is Kafka Streams?
 - 57. Difference between **Kafka Streams vs Spark Streaming**.
 - 58. What is **state store**?
 - 59. What is **KTable vs KStream**?
 - 60. What is Kafka Connect?
 - 61. Source vs Sink connectors.
 - 62. When would you use Kafka Connect instead of custom consumer?
 - 63. How does Connect handle failures?
-

8 Security & Governance (Advanced)

- 64. What security mechanisms does Kafka support?
 - 65. Explain **SSL vs SASL**.
 - 66. What is **ACL** in Kafka?
 - 67. How to secure Kafka in production?
 - 68. How does authorization work?
 - 69. What is Schema Registry?
 - 70. Avro vs JSON vs Protobuf.
-

9 Real-World Scenario Based Questions (Very Important)

- 71. How do you design Kafka for **banking transactions**?
- 72. How do you prevent **duplicate processing**?
- 73. How do you handle **poison messages**?
- 74. What if a consumer crashes after processing but before commit?
- 75. How to replay messages?
- 76. How to migrate data between clusters?

- 77. How do you handle schema evolution?
 - 78. How to guarantee ordering for a specific user?
 - 79. How to handle **burst traffic**?
 - 80. Kafka vs Pulsar – which one would you choose and why?
-

10 Kafka System Design Questions (Senior / Architect)

- 81. Design a **real-time log processing system** using Kafka.
 - 82. Design **payment event streaming** using Kafka.
 - 83. Design Kafka for **1 million messages/sec**.
 - 84. Multi-region Kafka design.
 - 85. How to do **disaster recovery** in Kafka?
 - 86. Active-active Kafka cluster design.
 - 87. How do you scale consumers horizontally?
 - 88. Kafka in microservices architecture.
 - 89. How do you handle backward compatibility?
 - 90. How do you decommission a broker safely?
-

1 Tricky / Follow-Up Questions

- 91. Can Kafka lose messages? When?
- 92. Why Kafka does not need random reads?
- 93. Can a consumer read from multiple topics?
- 94. What happens if ISR shrinks to zero?
- 95. Can two consumers read same partition?
- 96. Why Kafka prefers disk over memory?
- 97. What is the downside of exactly-once?
- 98. How Kafka differs from traditional JMS?
- 99. What is controller failover?
- 100. Biggest Kafka production issue you faced?